

Analysis: ESAb ATAd

Test Set 1

In Test Set 1, there are only 10 positions in the string. We can query for each of them and then submit the complete string, without having to worry about any quantum fluctuations (which would only happen if we submitted an 11th query).

Test Set 2

Here is one of various ways to solve the second test set. We begin by querying for the first ten positions in the real string, then create a "possibility set" containing all 1024 20-character strings that begin with those 10 characters. Then we update our "possibility set" to contain all strings that could have arisen from those strings after the next quantum fluctuation. The correct answer is in here somewhere — now we need to narrow the set down!

Before making each subsequent query, we first find the string index (between 1 and 20) at which the proportion of 0s and 1s among the strings in our possibility set is most nearly even. Then we query the real string at that index, and eliminate from the possibility set any strings that are not consistent with that information. Whenever we can indeed find a position with even proportions, we are guaranteed to cut the size of the set in half, but if there is no such position, we may not be able to eliminate that many possibilities. We can continue in this way, remembering to expand the possibility set every time there is a quantum fluctuation, until only one possibility remains, which must be the answer.

It is not easy to prove that this strategy will converge upon an answer. Intuitively, we can observe that a quantum fluctuation increases the size of the possibility set by at most 4, and even if we somehow only cut the possibility set by 20% with each pruning, we would still easily beat that factor-of-4 increase and make enough progress to finish within 150 queries. Moreover, it would not be possible for the strings in the possibility set to all be distinct while being so similar at every individual position (recall that we always pick the position that will be *most* useful to us in the worst case). Also, Test Set 2 is a Visible Verdict set, so we might as well just submit our answer and see.

Test Set 3

The above strategy will not work for 100-character strings, since the possibility set would be astronomically huge. Fortunately, there is a much simpler approach.

Observe that if we can find two positions that are equidistant from the center of the string and have the same value, we can use them to detect when a quantum fluctuation has included a complementation (with or without a reversal). Suppose, for example, that the two ends of the string are 0 just before a quantum fluctuation. After the fluctuation, we can check the first one. If it is 1, then there was a complementation; if not, there wasn't one. This is true regardless of whether that quantum fluctuation included a reversal.

Now suppose that we continue to check pairs of positions in this way, moving inward one step at a time. After every quantum fluctuation, we must spend one query to check for complementation so we can update our existing knowledge about the string if there has been one. If every pair turns out to be a "same pair" like the first pair, then we never needed to care about reversals anyway (since the string is palindromic), and we are done.

But what if, in the course of this, we find a "different pair"? Such pairs are helpful in their own way! If we query the first position of a "different pair" after a quantum fluctuation and we find that that bit has changed, then we know that either a complementation or reversal has happened, but not both.

Once we have such a "different pair", we can use it in conjunction with the "same pair", spending 2 out of every 10 queries to learn exactly what happened in each quantum fluctuation. For example, if the first position of our "same pair" stayed the same but the first position of our "different pair" did not, we know that the quantum fluctuation included a reversal but no complementation.

In the above analysis, we assumed we would encounter a "same pair" first. If the first pair is different, though, we can proceed until we encounter a "same pair"; if we never encounter one, then we do not care about the distinction between complementation and reversal, because the operations are equivalent for that particular string. If we do encounter a "same pair", though, then we can proceed as above.

How many queries will we need in the worst case? We can use all of our first 10 to gather data, since whatever happened in the quantum fluctuation at the start of the problem is unknowable and does not matter. After that, we may need to use up to 2 out of every 10 queries to reorient ourselves before spending the remaining 8 gathering data. So, to be sure we can find the entire string, we will need 10 queries, plus 11 more sets of 10 queries in which we learn 8 positions each time, (to get us to 98 positions known), plus 2 more queries for a final reorientation, plus 2 more to get the last two positions. That is a total of 124, which is well within the allowed limit of 150.

Regarding the name...

Last year, we had the [Dat Bae problem](#) about deletions from a string in a database; the name was `Data Base`, altered in a way that reflected the theme. `ESAb ATAd` is similar, with case change serving as a rough equivalent of complementation. (Imagine how much the Code Jam team has enjoyed trying to type the name correctly each time!)

Analysis: Indiciu

Test Set 1

There are a few different options for solving test set 1. Since there are only 44 possible cases, one option is to generate all answers by hand or via a program that is run locally, then submit a program that dispenses those. Another approach is to notice that there are not many different Latin squares for $N \leq 5$ (see the number of Latin squares [here](#)), and check them all. To generate all Latin squares, we can recursively fill in the cells one by one. For each cell, we try all N possible values. For each one, we ensure that it does not conflict with any cells in the same row or same column. Since there are at most 161280 Latin squares to consider, this is quite quick.

Test Set 2

Unfortunately, once N gets even slightly large, there are way too many Latin squares to generate them all (for $N = 11$, for example, there are 776966836171770144107444346734230682311065600000 different Latin squares).

There are many creative ways to solve this test set. The [Code Jam forum](#) is a good place to share and discuss different solutions! For example, we can directly create Latin squares with the appropriate trace by modifying structured Latin squares (for example, by modifying [circulant Latin squares](#)). Below, we discuss an easy-to-implement idea which is a little tricky to come up with and uses a graph algorithm in the middle!

First, we start by dealing with the impossible cases. If $K = N+1$, then the only possible diagonals have exactly one 2 and $N-1$ 1s. However, if $N-1$ of the diagonal elements are 1, then the only location for the 1 in the remaining row must be on the diagonal, so we cannot make a sum of $N+1$. Similarly, we cannot make a sum of N^2-1 since the only possible diagonal is one $N-1$ and $N-1$ N s.

We will now show a construction which works for every other case (with 2 additional small cases that don't work, see below). One of the main insights needed is that all possible sums are achievable using a diagonal with almost all values the same. In particular, we may assume that at least $N-2$ values are the same: AAAA ... AABC for some A, B, C (not necessarily all different).

For example, if $N = 10$ and $K = 20$, we can choose $A = 2$, $B = 2$, and $C = 2$. If $N = 10$ and $K = 55$, we can choose $A = 6$, $B = 4$, and $C = 3$. We already showed above that $A = B$ if and only if $A = C$. We leave it as an exercise to show that all values for K between N and N^2 are possible with these constraints. (Note: you have to be a little careful with $N = 3$. If $B = C$, then $A = B = C$ for a similar reason; so with $N = 3$, neither $K = 5$ nor $K = 7$ will have solutions). To find the appropriate values of A, B, and C, we can brute force all possible triples and check whether the chosen diagonal will work.

Now that we know what the diagonal looks like, how do we actually find a Latin square that has this diagonal? To do that, we will fill in the unfilled cells row by row. We will use [bipartite matching](#) to find a valid row. In one bipartition, we have N vertices for the N cells in that row. In the other bipartition, we have N vertices for the N numbers that can be placed into the cell. Make an edge between the cell vertex on the diagonal and the number vertex that was decided on. For every other cell, make an edge between a cell vertex and a number vertex if that number can be put into that cell without breaking the Latin square properties.

We can greedily pick any perfect matching for each row *starting with the rows with B and C on their diagonal*. Once we have filled in these two rows, we can use [Hall's Marriage Theorem](#) to show that we will never run into any issues (so long as the conditions above about A, B, C are met).

Hall's Theorem

This section is dedicated to proving the above claim that Hall's theorem holds. We will assume in this section that the reader is comfortable with Hall's theorem. A one sentence high-level reminder of Hall's theorem: *All subsets of one bipartition have a neighborhood that is at least as large as the original subset if and only if the graph has a perfect matching.*

For the explanation here, we will make the top two rows with B and C on their diagonal as the top two rows. We'll assume that these two rows are already filled in (and leave the proof you can do this to the reader). The important part is that the top-left 2×2 submatrix is $\begin{smallmatrix} CA \\ AB \end{smallmatrix}$. Now imagine that we have filled in $N-k$ rows (and have k mostly empty rows). Consider this example with $N = 8$ and $k = 3$. (? means filled in, but it doesn't matter with what and _ means not filled in yet):

```
CA?????
AB?????
??A????
???A????
????A???
_____A_____
_____A_____
_____A_____
```

For each of the $N-1$ non-A "cell vertices", the $N-k$ vertices on the left of the diagonal have a degree of k and the $k-1$ vertices on the right of the diagonal have a degree of $k-1$ (because the number A is also restricted). For each of the $N-1$ non-A "number vertices", each number originally had degree N and we have removed at least $N-k$ of those edges since the number appeared once in the top $N-k$ rows. Thus, the maximum degree of the "number vertices" is k .

We will ignore the "cell vertex" and the "number vertex" corresponding to the forced diagonal entry since that will be forced in our matching (and leaving it out makes our math below easier).

Let X be a subset of "cell vertices". Let $m = |X|$. We must show that $|N(X)| \geq m$ in order to utilize Hall's theorem (where $N(X)$ is the set of "number vertices" that are adjacent to at least one vertex in X). We have 2 separate cases:

Case 1: $m \leq k-1$.

Since the degree of each vertex in X is at least $k-1$, the number of edges leaving X is at least $m \times (k-1)$. Consider the "number vertices" that these edges are absorbed into. Since the maximum degree of "number vertices" is k , there are at least $(m \times (k-1))/k$ "number vertices" that absorb these edges. That is, $|N(X)| \geq (m \times (k-1))/k = m-m/k$. Since $m \leq k-1$, we have that $m/k < 1$. So $|N(X)| > m-1$. Since $|N(X)|$ is an integer, we have $|N(X)| \geq m$ as desired.

Case 2: $m \geq k$.

Consider the edges leaving X . At most $k-1$ of them have degree $k-1$, and the remaining have degree k . Thus, the number of edges leaving X is at least $(k-1) \times (k-1) + (m-(k-1)) \times k$. Since the maximum degree of "number vertices" is k , there are at least $((k-1) \times (k-1) + (m-(k-1)) \times k)/k$ "number vertices" that absorb these edges. That is, $|N(X)| \geq ((k-1) \times (k-1) + (m-(k-1)) \times k)/k$.

$(k-1) \times k/k = m - (1 - 1/k)$. Since $1 - 1/k < 1$, we have $|N(X)| > m-1$. Since $|N(X)|$ is an integer, we have $|N(X)| \geq m$ as desired.

Thus, in all cases, the conditions for Hall's theorem are satisfied, so there exists a perfect matching and we can iteratively complete the Latin square.

Analysis: Nesting Depth

Test Set 1

To solve Test Set 1, we can put an opening parenthesis before each group of 1s and a closing parenthesis after.

We can use the following trick to simplify the implementation: prepend and append one extra 0 to **S**. Then the implementation is just replacing 01 with 0(1 and 10 with 1)0, which can be written in one line of code in some programming languages. Don't forget to remove the extra 0s from the end of the resulting string!

Test Set 2

For convenience, let's once again use the trick described above: prepend and append extra 0s to **S**, and then scan **S** from left to right.

Suppose we see some number A immediately followed by some larger number B and suppose all of the previously inserted parentheses would leave A at the right nesting depth — that is, there are exactly A unmatched opening parentheses preceding A, and no unmatched closing parentheses. For B to be at nesting depth B we need to add at least B - A opening parentheses. We can just do that and nothing else, to keep the final string length minimal. Any additional opening parentheses we would add would need to be closed before B, which would needlessly lengthen the string. Similarly, if we see some number A immediately followed by some smaller number B, we can just insert A - B closing parentheses. And in the case when A is equal to B, we don't need to add anything.

We don't need any parentheses before the temporary 0 in the beginning, or after the one in the end, so we can just drop them before printing the result.

Since we only add p parentheses when at least p are needed, the resulting string is of minimum length.

An inefficient but fun solution

The problem can be solved using only string replacements. First, replace each digit D with D (s, then the digit itself, then D)s. Then eliminate all instances of) (, collapsing the string each time, until there are no more to remove.

Here's a Python3 implementation:

```
for C in range(int(input())):
    rawstr = ''.join([int(x) * '(' + x + ')' * int(x) for x in str(input())])
    for _ in range(9):
        rawstr = rawstr.replace(') (', '')
    print("Case #{}: {}".format(C+1, rawstr))
```

Analysis: Parenting Partnering Returns

Test Set 1

We can solve this test set by naively trying every possible subset of activities to be covered by Jamie and assign the rest of the activities to be covered by Cameron. For each subset of activities, we can check whether a pair of activities overlap for each pair of activities. An activity with start time s_1 and end time t_1 overlaps with another activity with start time s_2 and end time t_2 if the time intersection is not empty (i.e., $\max(s_1, s_2) < \min(t_1, t_2)$).

The running time of this solution is $O(2^N \times N^2)$, which is fast enough to solve Test Set 1.

Test Set 2

We can solve this test set by greedily assigning the activities in increasing order of start time. For each activity (in increasing order of start time), we can check whether Jamie or Cameron can be assigned to cover the activity and assign the activity to whomever can be assigned to (or arbitrarily if both partners can be assigned). The check can be done by iterating all activities that have been previously assigned to Jamie and Cameron.

The greedy assignment is correct because the only way that the assignment fails is when there is a time that is covered by three activities. In such a case, there is indeed no valid assignment. When deciding who to assign an activity with start time s , only activities with start times no later than s have been assigned. Therefore, if both Jamie and Cameron have some activity assigned with end time later than s , it means that there are three activities that use the time between s and $s + 1$, and therefore, there is no possible assignment. If an assignment is possible, there cannot be any set of three activities that pairwise overlap, so by the contrapositive of the previous argument, we will be able to assign the activity to at least one of Jamie or Cameron at every step.

The running time of this solution is $O(N^2)$, which is fast enough to solve this test set. To optimize the solution to $O(N \log N)$ time, we can efficiently check whether an activity can be assigned to Jamie or Cameron by keeping track of the end time of the last activity assigned to each partner and comparing this to the start time of the new activity. In this case, only $O(N)$ extra time is needed after sorting the activities by their start time.

Graph approach

Another possible approach to solve this test set is to construct a [graph](#) with N nodes, each representing one activity. We add an edge connecting a pair of nodes if the pair of activities represented by the nodes overlap (see Test Set 1 section for details on how to check if two intervals overlap). This graph is commonly known as an [interval graph](#).

Therefore, the problem is equivalent to [finding a partition of nodes C and J such that every edge connects a node in C and a node in J](#), as we can assign all activities represented by nodes in C to Cameron and all activities represented by nodes in J to Jamie. The running time of the algorithm to find the partition (or report if one does not exist) is linear on the size of the graph. The graph has N nodes and $O(N^2)$ edges, which means the solution requires $O(N^2)$ time to build the graph and $O(N^2)$ time to run the partition algorithm, so also $O(N^2)$ time overall.

Analysis: Vestigium

One simple way to check whether the values in a row or column are a permutation of the values from 1 to N is to sort them and then step through them, checking whether the sorted list starts at 1 and increases by 1 each time. Another option, which avoids the sort and takes time linear in N , is to look at the values one by one and store each one in a hash table-based data structure. If we ever find that a value is already in the set, then that row or column contains a repeated value. Because there are N values and the problem guarantees that they are integers between 1 and N , inclusive, the absence of duplicates implies that we have a permutation as desired.

Finding the trace is also straightforward — iterate through the rows taking the i -th value from the i -th row, and add the values together.